vue相关面试题

1. vue的优缺点

```
优点：性能好，简单易用，前后端分离，双向数据绑定，单页面应用用户体验好。
缺点：不利于SEO优化。
```

2.vue中如何获取DOM元素

```
（1）通过给元素绑定ref="XXX"，

（2）通过this.$refs.XXX 或者 this.$refs['XXX']来获取

* 注意：vue中操作dom需要谨慎，尤其是添加或者删除dom的时候，
```

3.vue-loader是什么？使用它的用途有哪些？

```
vue-loader是解析 .vue 文件的一个加载器，将template/js/style转换成js模块。
js可以使用ES6语法，style样式可以用scss或者less，template 可以加jade等。
```

4.Vue 中为什么使用key？

```
用key来给每个节点做一个唯一标识，Diff算法就可以正确的识别此节点，找到正确的位置区插入新的节点，
可以高效的更新虚拟DOM。
```

5.axios解决跨域

```
1 配置 BaseUrl，配置访问的URL前缀。

import Vue from 'vue'
import App from './App'
import Axios from 'axios'

Axios.defaults.baseURL = '/api'
// 每次发送请求都回带一个 /api 的前缀。


2 配置代理，在src文件夹下创建文件 vue.config.js文件。添加一下内容。
module.exports = {
  outputDir: 'dist', // build 输出目录
  assetsDir: 'assets', // 静态资源目录
  lintOnSave: false, // 是否开启eslint
  devServer: {
    open: true, //是否自动弹出浏览器页面
    host: 'localhost',
    port: '8080',
    https: false, // 是否使用 HTTPS协议
    hotOnly: false, // 是否开启热更新
    proxy: {
      '/api': {
        target: 'https://www.xxx.com/api', // api服务器的地址
```

```
        changeOrigin: true,
        pathRewrite: {
          '^/api': ''
        }
      }
    }
  }
}


3 修改请求的URL，

// 假设请求地址为 https://www.xxx.com/api/deatils/info
this.$axios.get('/details/info')
  .then()
  .catch()
```

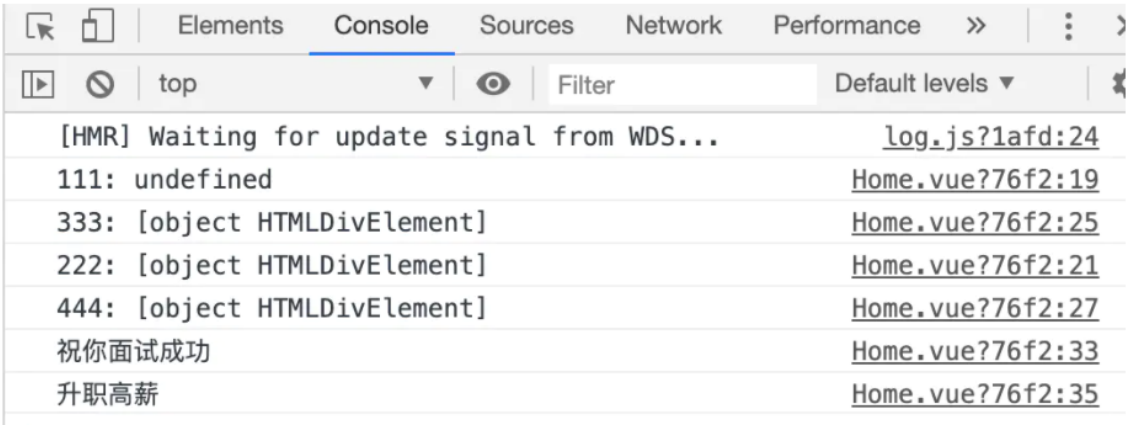
6. $nextTick() 的使用

```
this.$nextTick()将回调延迟到下次DOM更新循环之后执行。在修改数据之后立即使用他，然后等待dom更新。他跟全局的Vue.nextTick一样，不同的是回调的this自动绑定到调用他的实例上。

<template>
  <div>
    <div ref="wrap">{{value}}</div>
    <button @click="clickBtn">点击改变</button>
  </div>
</template>

<script>
export default {
  name: 'home',
  components: {
  },
  data() {
    return {
      value: '祝你面试成功'
    }
  },
  created() {
    console.log(`111: ${this.$refs['wrap']}`)
    this.$nextTick(() => {
      console.log(`222: ${this.$refs['wrap']}`)
    })
  },
  mounted() {
    console.log(`333: ${this.$refs['wrap']}`)
    this.$nextTick(() => {
      console.log(`444: ${this.$refs['wrap']}`)
    })
  },
  methods: {
    clickBtn() {
      this.value = '升职高薪'
      console.log(this.$refs['wrap'].innerText)
      this.$nextTick(() => {
```

```
        console.log(this.$refs['wrap'].innerText)
      })
    }
  }
}
</script>
```



根据打印顺序可以看到，在created()执行的时候DOM并未进行任何渲染，而此时进行的DOM操作没作用，而在created()里面使用this.$nextTick()可以等待DOM生成以后再来获取dom对象。

根据上面的放马点击栗子可以看出，在方法里直接打印的话，由于dom元素还没有更新，因此打印出来的还是为改变之前的值，而通过this.$nextTick()获取到的值为dom更新之后的值。

this.$nextTick()在页面交互，尤其是从后台获取数据后重新生成dom对象之后的操作有很大的优势。

7. vue组件中的data为什么必须是一个函数？

如果不用函数的话，实例化的组件是共享同样的一个data对象，当你修改一个属性的时候，data也恢复阿生改变。
当data是一个函数的时候，每个实例的data属性都是独立的，不会相互影响。

8.Vue中双向数据绑定是如何实现的？（vue双向数据绑定原理）

vue实现双向数据绑定的原理就是利用了Object.defineProperty()这个方法重新定义了对象获取属性值get()和设置属性值set()的操作来实现的。

```
// Object.defineProperty()方法
let obj = {}
let name
Obejct.defineProperty(obj,'name',{
  get: function() {
    console.log('获取')
    return val
  }
  set: function(newVal) {
    console.log('设置')
    }
})
obj.name = 'arthas' // 在给obj设置name属性的时候，触发了set这个方法
const val = obj.name // 在得到obj的name属性，会触发get方法。
```

```html
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="ie=edge">
  <title>Document</title>
</head>
<body>
  <div id="test"></div>
  <input type="text" id="text">
  <script>
    const obj = {}
    const test = document.querySelector('#test')
    const text = document.querySelector('#text')
    Object.defineProperty(obj,'name',{
      get: function() {
        return val
      },
      set: function(newVal) {
        text.value = newVal
      }
    })
    text.addEventListener('input', function(e){
      // 给obj的name属性赋值，进而触发该属性的set方法
      obj.name = e.target.value
    })
    obj.name = 'arthas' // 再给obj设置name属性的时候，触发了set这个
    test.innerHTML = obj.name // 调用obj的get属性获取name
  </script>
</body>
</html>
```

9.vue-router 路由模式有几种？

```
vue-router 有 3 种路由模式：hash、history、abstract

hash: 使用 URL hash 值来作路由。支持所有浏览器，包括不支持 HTML5 History Api 的浏览器；
history : 依赖 HTML5 History API 和服务器配置。具体可以查看 HTML5 History 模式；
abstract : 支持所有 JavaScript 运行环境，如 Node.js 服务器端。如果发现没有浏览器的 API，
路由会自动强制进入这个模式.
```

10 能说下 vue-router 中常用的 hash 和 history 路由模式实现原理吗？

```
（1）hash 模式的实现原理
早期的前端路由的实现就是基于 location.hash 来实现的。其实现原理很简单，location.hash 的值
就是 URL 中 # 后面的内容。比如下面这个网站，它的 location.hash 的值为 '#search'：
https://www.word.com#search

hash 路由模式的实现主要是基于下面几个特性：
URL 中 hash 值只是客户端的一种状态，也就是说当向服务器端发出请求时，hash 部分不会被发送；
hash 值的改变，都会在浏览器的访问历史中增加一个记录。因此我们能通过浏览器的回退、前进按钮控制
hash 的切换；
可以通过 a 标签，并设置 href 属性，当用户点击这个标签后，URL 的 hash 值会发生改变；或者使用
JavaScript 来对 loaction.hash 进行赋值，改变 URL 的 hash 值；
我们可以使用 hashchange 事件来监听 hash 值的变化，从而对页面进行跳转（渲染）。
```

（2）history 模式的实现原理
HTML5 提供了 History API 来实现 URL 的变化。其中做最主要的 API 有以下两个：history.pushState() 和 history.repalceState()。这两个 API 可以在不进行刷新的情况下，操作浏览器的历史纪录。唯一不同的是，前者是新增一个历史记录，后者是直接替换当前的历史记录，如下所示：

```
window.history.pushState(null, null, path);
```

history 路由模式的实现主要基于存在下面几个特性：

pushState 和 repalceState 两个 API 来操作实现 URL 的变化 ；

我们可以使用 popstate 事件来监听 url 的变化，从而对页面进行跳转（渲染）；

history.pushState() 或 history.replaceState() 不会触发 popstate 事件，这时我们需要手动触发页面跳转（渲染）。

11 什么是 MVVM？

odel-View-ViewModel （MVVM） 是一个软件架构设计模式，由微软 WPF 和 Silverlight 的架构师 Ken Cooper 和 Ted Peters 开发，是一种简化用户界面的事件驱动编程方式。由 John Gossman（同样也是 WPF 和 Silverlight 的架构师）于2005年在他的博客上发表

MVVM 源自于经典的 Model-View-Controller（MVC）模式 ，MVVM 的出现促进了前端开发与后端业务逻辑的分离，极大地提高了前端开发效率，MVVM 的核心是 ViewModel 层，它就像是一个中转站（value converter），负责转换 Model 中的数据对象来让数据变得更容易管理和使用，该层向上与视图层进行双向数据绑定，向下与 Model 层通过接口请求进行数据交互，起呈上启下作用。如下图所示：

图片

> **（1）View 层**
>
> View 是视图层，也就是用户界面。前端主要由 HTML 和 CSS 来构建 。
>
> **（2）Model 层**
>
> Model 是指数据模型，泛指后端进行的各种业务逻辑处理和数据操控，对于前端来说就是后端提供的 api 接口。
>
> **（3）ViewModel 层**
>
> ViewModel 是由前端开发人员组织生成和维护的视图数据层。在这一层，前端开发者对从后端获取的 Model 数据进行转换处理，做二次封装，以生成符合 View 层使用预期的视图数据模型。

> 需要注意的是 ViewModel 所封装出来的数据模型包括视图的状态和行为两部分，而 Model 层的数据模型是只包含状态的，比如页面的这一块展示什么，而页面加载进来时发生什么，点击这一块发生什么，这一块滚动时发生什么这些都属于视图行为（交互），视图状态和行为都封装在了 ViewModel 里。这样的封装使得 ViewModel 可以完整地去描述 View 层。
>
> MVVM 框架实现了双向绑定，这样 ViewModel 的内容会实时展现在 View 层，前端开发者再也不必低效又麻烦地通过操纵 DOM 去更新视图，MVVM 框架已经把最脏最累的一块做好了，我们开发者只需要处理和维护 ViewModel，更新数据视图就会自动得到相应更新。
>
> 这样 View 层展现的不是 Model 层的数据，而是 ViewModel 的数据，由 ViewModel 负责与 Model 层交互，这就完全解耦了 View 层和 Model 层，这个解耦是至关重要的，它是前后端分离方案实施的重要一环。

12Vue 是如何实现数据双向绑定的？

Vue 数据双向绑定主要是指：数据变化更新视图，视图变化更新数据，如下图所示：

即：

- 输入框内容变化时，Data 中的数据同步变化。即 View => Data 的变化。
- Data 中的数据变化时，文本节点的内容同步变化。即 Data => View 的变化。

其中，View 变化更新 Data ，可以通过事件监听的方式来实现，所以 Vue 的数据双向绑定的工作主要是如何根据 Data 变化更新 View。

Vue 主要通过以下 4 个步骤来实现数据双向绑定的：

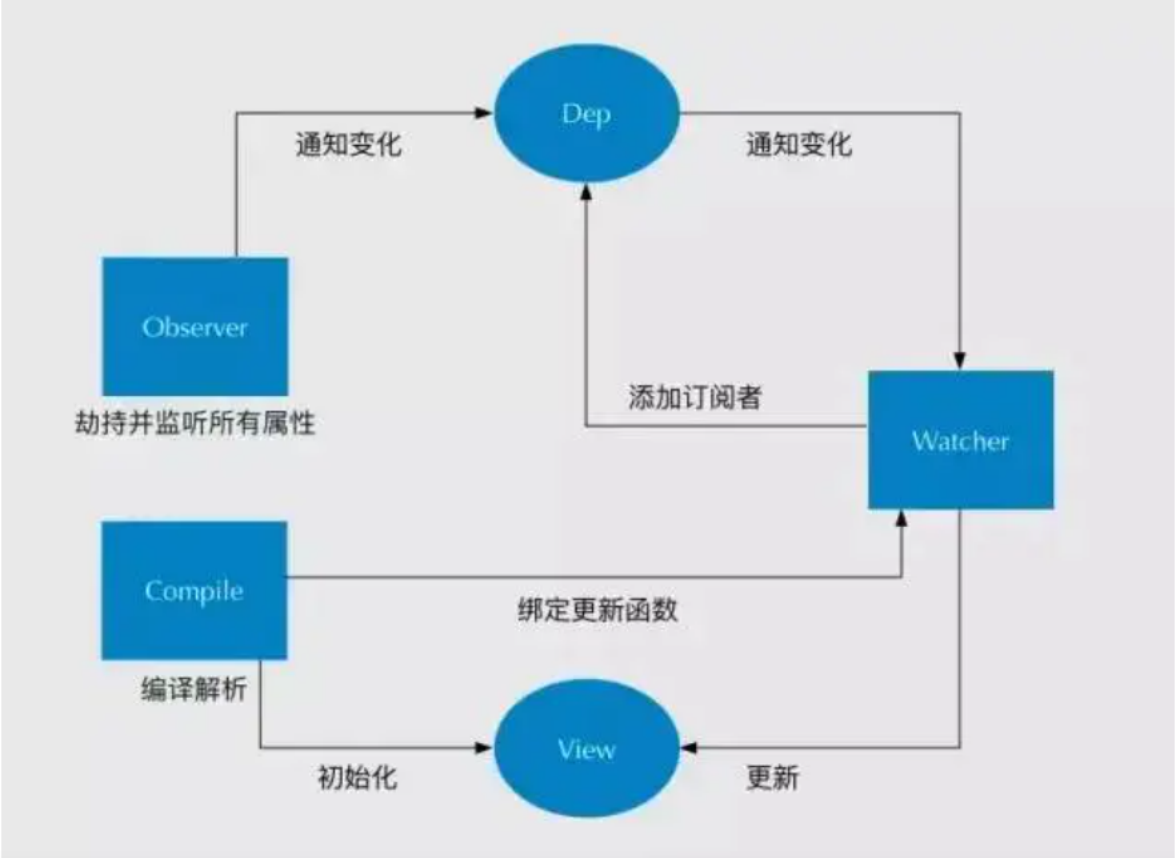
实现一个监听器 Observer：对数据对象进行遍历，包括子属性对象的属性，利用 Object.defineProperty() 对属性都加上 setter 和 getter。这样的话，给这个对象的某个值赋值，就会触发 setter，那么就能监听到了数据变化。

实现一个解析器 Compile：解析 Vue 模板指令，将模板中的变量都替换成数据，然后初始化渲染页面视图，并将每个指令对应的节点绑定更新函数，添加监听数据的订阅者，一旦数据有变动，收到通知，调用更新函数进行数据更新。

实现一个订阅者 Watcher：Watcher 订阅者是 Observer 和 Compile 之间通信的桥梁 ，主要的任务是订阅 Observer 中的属性值变化的消息，当收到属性值变化的消息时，触发解析器 Compile 中对应的更新函数。

实现一个订阅器 Dep：订阅器采用 发布-订阅 设计模式，用来收集订阅者 Watcher，对监听器 Observer 和 订阅者 Watcher 进行统一管理。

以上四个步骤的流程图表示如下，如果有同学理解不大清晰的，可以查看作者专门介绍数据双向绑定的文章《0 到 1 掌握：Vue 核心之数据双向绑定》，有进行详细的讲解、以及代码 demo 示例。



13虚拟 DOM 的优缺点？

优点：

保证性能下限： 框架的虚拟 DOM 需要适配任何上层 API 可能产生的操作，它的一些 DOM 操作的实现必须是普适的，所以它的性能并不是最优的；但是比起粗暴的 DOM 操作性能要好很多，因此框架的虚拟 DOM 至少可以保证在你不需要手动优化的情况下，依然可以提供还不错的性能，即保证性能的下限；

无需手动操作 DOM： 我们不再需要手动去操作 DOM，只需要写好 View-Model 的代码逻辑，框架会根据虚拟 DOM 和 数据双向绑定，帮我们以可预期的方式更新视图，极大提高我们的开发效率；

跨平台： 虚拟 DOM 本质上是 JavaScript 对象,而 DOM 与平台强相关，相比之下虚拟 DOM 可以进行更方便地跨平台操作，例如服务器渲染、weex 开发等等。

缺点：

无法进行极致优化： 虽然虚拟 DOM + 合理的优化，足以应对绝大部分应用的性能需求，但在一些性能要求极高的应用中虚拟 DOM 无法进行针对性的极致优化。

14 虚拟 DOM 实现原理？

虚拟 DOM 的实现原理主要包括以下 3 部分：

用 JavaScript 对象模拟真实 DOM 树，对真实 DOM 进行抽象；

diff 算法 — 比较两棵虚拟 DOM 树的差异；

pach 算法 — 将两个虚拟 DOM 对象的差异应用到真正的 DOM 树。

15 你有对 Vue 项目进行哪些优化？

（1）代码层面的优化

```
v-if 和 v-show 区分使用场景
computed 和 watch 区分使用场景
v-for 遍历必须为 item 添加 key，且避免同时使用 v-if
长列表性能优化
事件的销毁
图片资源懒加载
路由懒加载
第三方插件的按需引入
优化无限列表性能
服务端渲染 SSR or 预渲染

（2）Webpack 层面的优化
Webpack 对图片进行压缩
减少 ES6 转为 ES5 的冗余代码
提取公共代码
模板预编译
提取组件的 CSS
优化 SourceMap
构建结果输出分析
Vue 项目的编译优化

（3）基础的 Web 技术的优化
开启 gzip 压缩
浏览器缓存
CDN 的使用
使用 Chrome Performance 查找性能瓶颈
```